

1. What I'm Building Today

A gaming system for coders, developers and engineers with integrated interview challenges unlocked after correctly answering several challenge questions.

Project: Dev Play

Today's goal: Is to create website/app, test challenges and verify challenge source.

2. User Stories

Write 3-5 user stories for what you're building today. Format: "As a [who], I want to [what] so that [why]."

#	User Story	Priority
1	As a _coders, developers, engineers__, I want to _create the platform__ so that _I can test it__	Test the challenges and buttons
2	As a _reviewer__, I want to _make sure that challenges are sound__ so that _there is no misinformation_	Check sources
3	As a _developer__, I want to _include more animation and feedback__ so that _it is helpful in the interview process._	Nice to more animations
4	As a ____, I want to ____ so that ____	Nice to Have
5	As a ____, I want to ____ so that ____	Stretch

3. Backlog

Break your user stories into specific tasks. Be concrete — "make it look good" is not a task. "Add flexbox layout to the card grid" is.

Task	From Story #	Priority	Estimate	Status
Use a rating system		Must Do	S	Started
Include 25 challenges		Must Do	S	Started

Use colors from Google logo, but no white background		Must Do	S	Started
2 interview docs given with a screenshot as interview examples		Nice to Have	S	Started
Challenge sources need to be revised, AI made up 27 out of 29 challenges		Nice to Have 500 challenges	L	Started but need to verify each challenge. Need to give AI stronger constraints. AI cited sources for reliable challenges.
Make sure that the browser's built-in Web Speech API — specifically <code>webkitSpeechRecognition</code> works.		Stretch	L	Not Started

Estimates: S = under 30 min, M = 30–60 min, L = over 1 hour

Rule: If your Must Do tasks total more than ~3 hours of estimated work, you have too many Must Dos. Move something to Nice to Have.

4. Definition of Done

How do you know you're finished? Be specific.

- [I know I am finished when the task works the way I planned it to work]
- [I know I am finished when it passes a test.]
- [I know I am finished after I have refined it and made changes to improve it to the best of my ability.]
- Code committed and pushed to GitHub: Even after I push it to GitHub, I run into issues, and I know that I am not finished when new problems surface.

5. End of Day Retro

Fill this out at the fireside chat.

What I shipped today:

What I learned:

What blocked me:

What I'd do differently next Friday:

My sprint score: ____/10 (*Did I finish what I planned? 10 = shipped everything. 1 = nothing went to plan.*)

6. The prompts that I used to create this app:

Prompt 1:

- Create a developer playground to learn code with test examples, and a rating system that rates their ability. give the developer options to chose from that will help them solve the code challenge.
- Include the Fibonacci challenge, the palindrome challenge and 25 other code challenges.
- The developer playground should have vibrant colors like Google's logo, chose a background color, modern style. use large font and small font, use shadow box, hover effect, flex, slide out and fade in features.
- Utilize scroll to go to the next challenge. Use HTML with inline CSS and JS.
- Include links at the top: hint, next challenge, player scores against other coders success challenges.
- Include trophy levels as the player completes different levels of challenges such as blue dragon, red dragon, yellow dragon, blue ribbon, red ribbon, green ribbon, bronze metal, silver metal, gold metal, small trophy, medium trophy, large trophy, extra large trophy for each level.

Prompt 2:

- Update the code and and allow user to click play and get the first of 27 codes. display the code on the screen and for easy challenges, allow the user to select from codes to complete the code challenge. After 5 wins give the user a trophy.
- Organize the trophies by wins and difficulty level. make the scroll down for next challenge a button to allow the user to move to the next challenge, if they chose. display the next challenge after the user clicks the scroll down to the next challenge.
- The green button that says solved! +100pts move it to the right of the button next to the scores.
- Below the buttons include a score window of challenges completed. Use white font for the code displayed on the screen. Revise the code and update.

Prompt 3:

- I would like to incorporate an interview rating system with pulsating hearts, to integrate and add to my coding game Dev Play, I would like to include a section of interview questions for coding gamers to select from, like the one I had, which was to write a js code and decode the message.
- The challenge was to decode a url, the undecoded message attached was in a url link: This was my js code to decode it: I want to expand on my Dev Play coding game and add new features suc as

these types of challenges, including voice recognition to test players on explaining logic and answering interview questions regarding coding and software engineering and development.

- How would you integrate that based on the Dev Play code you revised?
- Where would you add the interview options, would you have the player click a link above or below and scroll down to the next viewport to preview interviewing based on their level of wins? Walk me through your idea and show me with code how you would apply this concept.

Me: How many interview questions are there?

Me: I would like to know more about the voice explainer? how does it take in information, is ai being used to translate and find out if the explanation was correct and if so how?

AI's Answer: **How the Voice Explainer Works Right Now**

The voice input uses the browser's built-in **Web Speech API** — specifically `webkitSpeechRecognition`. This is a native browser feature, not AI you're paying for or calling externally. Here's the pipeline:

Your voice → browser mic → Web Speech API → text transcript → keyword scanner → heart score

Step 1 — Capture: When you press the mic button, the browser opens your microphone and streams audio to Google's speech-to-text servers (built into Chrome). This happens automatically — no API key, no setup.

Step 2 — Transcribe: Chrome returns a rolling text transcript as you speak, word by word in real time. That's what appears in the transcript box.

Step 3 — Score: When you stop speaking, the code scans your transcript for a predefined list of keywords I hardcoded. For example, the Two Sum question expects words like hash, map, complement, $O(n)$, linear, lookup. It counts how many you hit versus the minimum required and converts that ratio into a 1–5 heart score.

Prompt 4:

Can you include an option where the user gets to choose python, JavaScript in the interview questions.

- A toggle that allows the interviewer to choose code language underneath the Decode challenge, voice explainer and concept Q&A?
- Can you make the Dev Play clickable to bring the user to the home page?
- Can you find coding interview questions on the internet from reliable sources that interview how to set up a full-stack app with back end and include it in the interview challenge?
- And interview questions for front-end and interview challenges for back-end? Do these questions deserve their own category?
- If so, where, if not explain what category these interview challenges fall under?
- Can you add buttons under decode challenge, voice explainer, and concept Q&A, if you do add a category for full-stack, front-end and back end, list 10 questions each.

- Can you also include interview challenge questions and category AI prompting, let the challenges be multiple choice code display of 1 wrong AI code and 1 correct, and another challenge of 2 wrong AI codes and 1 correct, of an all of the above option (3 AI correct codes).
- Explain, that the AI was prompted to code and these were the results: AI prompting coding challenges, select the correct code and give the user the multiple choice.

Prompt 5:

- 2 of the challenge questions of the 29 questions in the beginning are from documents I gave you.
- They are from reliable sources. Are the other 27 questions from reliable sources if so can you site them.
- Also, can you add more challenges and make it 500 challenges. Is that too much for you to handle?
- Given the constraints of getting the challenges from reliable sources?
- If not, build this. Reliable sources are typically from .edu sources, companies interviewing, reliable coding organizations, MDN, W3 Schools, Google web dev, are some examples.

AI: Posed a challenge when building 500 questions and reduced it to 200 because I have all inline coding in one html, and said there would be a parsing challeng: parsing timing would take 3-6 seconds.

Me: What if we create three files, HTML, JS and CSS, would that matter in respect to the 800KB-1.2MB and parsing timing?

AI: Yes and explained why.

- **What actually does the parsing:**
- The **HTML Parser** reads the HTML document and builds the **DOM** (Document Object Model) — the tree of elements your JavaScript can interact with.
- The **CSS Parser** reads your stylesheet and builds the **CSSOM** (CSS Object Model) — a parallel tree of style rules.
- The **JavaScript Engine** (V8 in Chrome) reads and executes your JS. This is separate from both parsers.
- **What I should have said more precisely:**
- When everything is in one HTML file, the HTML parser hits the <style> block and hands it to the CSS parser — the HTML parser waits. Then it hits the <script> block, hands it to the JS engine, and **blocks the DOM from being built further** until the JS finishes executing. That blocking of DOM construction is the real performance cost.
- With defer, the JS downloads in the background but the JS engine doesn't execute it until the **DOM is fully built**. That's the key — defer waits for the HTML parser to finish constructing the DOM before handing control to the JS engine.
- So to directly answer your question — yes, I meant the DOM construction process specifically, not just "the browser" as a vague whole. The DOM is the precise thing being blocked.

Me: Let's split the files into three files, HTML, JS, and CSS, go ahead and build.

To get from 200 questions to 500 questions, AI collaborated with me to add new categories with citations:

The realistic plan to reach 500:

The 200 challenges already cover the core of each category. To reach 500 with genuinely sourced content we need to expand into new territory. Here's what I'm proposing to add — all from the same reliable sources (OWASP, Harvard, Python.org, MDN, CodeCamp testing curriculum, Typescript docs, Atlassian tutorials, etc.) :

Category	Challenges	Source
◆ TypeScript	25	typescriptlang.org
🌿 Git & CLI	20	Atlassian Git Tutorial + GitHub Docs
🔒 Web Security	20	OWASP Top 10 2021
● Node.js	25	Node.js + Express.js official docs
📊 Data Structures	30	CS50 + freeCodeCamp DSA
🐍 Python Advanced	20	Python.org docs
⚡ JS Advanced	25	MDN Web Docs
🎨 CSS Advanced	20	MDN CSS Reference
🌐 HTTP & APIs	15	MDN HTTP + REST conventions
🧪 Testing	20	Jest docs + freeCodeCamp
🧩 Extra Algorithms	80	LeetCode classics
Total new	300	

Prompt 6:

- Testing the question. Some questions don't work. In order for me to show you which questions don't work, I need the wrong answers to be in red at the top, that way as I test it, any question that doesn't work will not light up. Once we get every question working. I can show you.
- Used the Developer console to show AI the problem lines and debug.

Me: Revising challenges from reliable sources:

SQL the updated challenges -- source table:

Questions#	Challenge	Difficulty	Source
166	SELECT Statement	Easy	W3Schools SQL
167	WHERE Clause	Easy	W3Schools SQL
168	ORDER BY	Easy	W3Schools SQL
169	INSERT INTO	Easy	W3Schools SQL
170	UPDATE Statement	Easy	W3Schools SQL
171	DELETE Statement	Easy	W3Schools SQL

172	COUNT()	Easy	W3Schools SQL
173	GROUP BY	Medium	W3Schools SQL
174	HAVING Clause	Medium	W3Schools SQL
175	INNER JOIN	Medium	SQLite docs: sqlite.org/lang_select.html
176	LEFT JOIN	Medium	SQLite docs: sqlite.org/lang_select.html
177	DISTINCT	Easy	SQLite docs: sqlite.org/lang_select.html
178	LIKE Operator	Easy	SQLite docs: sqlite.org/lang_expr.html
179	IN Operator	Easy	SQLite docs: sqlite.org/lang_expr.html
180	NULL Values	Easy	SQLite docs: sqlite.org/nulls.html
181	BETWEEN Operator	Easy	SQLite docs: sqlite.org/lang_expr.html
182	AVG() Function	Easy	SQLite docs: sqlite.org/lang_aggfunc.html
183	PRIMARY KEY	Medium	SQLite docs: sqlite.org/lang_createtable.html
184	FOREIGN KEY	Medium	SQLite docs: sqlite.org/foreignkeys.html
185	SQL Aliases	Easy	SQLite docs: sqlite.org/lang_select.html

Prompt 7. Refining the game.

Let's also add a Prev button for the interview level up, there's only a next button but not a prev option for the user to return to the previous challenge if they choose. Added a Reset button because the browser remembers where the user left off.

Testing and Verification:

Three things fixed:

Problem	Root Cause	Fix
Dragon didn't slide out at 5 wins	First WIN_TROPHIES threshold was 25 — nothing fires until 25 wins	Added 🐉 Blue Dragon at 5 wins as the first milestone. Trophy order is now: 5 → 25 → 50 → 100 → 150 → 200 → 300 → 400 → 500
Trophy cards not lighting up	Duplicate .tr-item / .tr-section / .earned-tag CSS — second definition overriding first with slightly different values	Removed all 14 duplicate trophy CSS lines
Toast animation broken	Duplicate .tu-toast CSS block — second one overriding the animation	Removed the duplicate, keeping the single clean definition